



Natural language processing course

Angel Naumov, Naum Spasev, Marija Koshtrevska

Abstract

Large language models struggle to provide accurate, up-to-date answers in rapidly evolving domains such as competitive gaming, where patch cycles frequently invalidate prior knowledge. This project presents a domain-specific question-answering system for League of Legends that combines Retrieval-Augmented Generation (RAG) with a fine-tuned language model. We index three complementary data sources — patch notes, champion descriptions, and Diamond-tier ranked statistics — and retrieve relevant context at query time to ground model responses in current game state. Additionally, we fine-tune a LLaMA-3-8B model on Diamond-tier match and build data to further adapt it to high-level competitive play patterns. We evaluate four system configurations (base LLM, fine-tuned LLM, base+RAG, fine-tuned+RAG) using both automated and human evaluation on 30 domain-specific questions. Results show that RAG is the dominant contributing factor. Both RAG-augmented systems substantially outperform all non-retrieval baselines, including the larger reference model (LLaMA-3-70B), confirming that domain adaptation through retrieval and fine-tuning is more effective than model scale alone for specialized, rapidly-changing domains.

Keywords

Domain-Specific AI, Natural Language Processing, Retrieval-Augmented Generation, Game Analytics

Advisors: Slavko Žitnik

Introduction

In recent years, large language models (LLMs) have demonstrated impressive abilities in understanding and generating human-like text across various fields. However, despite their versatility, these models struggle with tasks requiring the latest specialized knowledge or a deep understanding of context.

These limitations are noticeable in competitive gaming environments, such as League of Legends. This complex multiplayer game has a continuously shifting strategic landscape due to frequent patch updates. The choices players make are based on a complex interaction of various factors, including the strengths and weaknesses of the champion, the items chosen, and past performance trends. This is something that a model based on statistics struggles to keep up with.

This work addresses these challenges by developing a domain-specific conversational assistant based on a RAG framework. Rather than relying on knowledge encoded during pretraining, the system dynamically retrieves relevant information from a tailored knowledge base containing patch notes, champion statistics, and match history data. This context is then incorporated directly into response generation. This design is particularly well-suited to League of Legends: when a new patch is released, only the knowledge base requires

updating, not the model itself. The result is a system that can accurately answer nuanced questions about champion performance, recent balance changes, and matchup recommendations, surpassing the capabilities of standalone LLMs.

Related Work

Earlier machine learning studies in League of Legends have mostly concentrated on match outcome prediction and player performance assessment, attaining robust outcomes through gradient boosting, tensor factorization, and event-driven win probability models [1, 2, 3]. Jiang et al. [4] notably demonstrated that incorporating patch version context directly into predictive models improves accuracy. This confirms that game knowledge is inherently version-dependent and rapidly evolving.

However, these models primarily operate on structured data and lack mechanisms for interactive knowledge access. This has led to growing interest in NLP approaches.

Lee et al. [5] introduced a voice-based large language model companion for beginner League of Legends players in their 2026 study, LeagueBot, reporting significant reductions in cognitive challenge and perceived tension in a controlled experiment with 33 players. However, participants reported

receiving recommendations for nonexistent items. The authors attributed this hallucination problem to the LLM’s lack of access to current game data. RAG was explicitly proposed as a solution, but latency concerns in fast-paced gameplay prevented its implementation. Motivated by these limitations, this work presents an RAG-based assistant for League of Legends that retrieves current patch notes, champion statistics, and match data, making it possible to provide practical, up-to-date responses.

Data

We combine three complementary data sources: (i) official Riot Games balance patch notes, (ii) per-match ranked data fetched from the official Riot Games Developer API[6], and (iii) aggregated champion build and matchup statistics scraped from u.gg’s public stats CDN. Each source plays a distinct role in the RAG knowledge base: patch notes describe *what* changed, while the u.gg aggregates and Riot match data describe *how* the game is actually being played at high ELO under those changes.

Patch Notes

Patch notes represent the primary and most critical component of the RAG knowledge base. Riot Games releases balance updates approximately every two weeks, introducing champion buffs, nerfs, item changes, and systemic adjustments that fundamentally alter the competitive landscape. Each patch document is chunked, embedded, and indexed into the vector store, allowing the retrieval pipeline to surface the most relevant balance changes in response to user queries. This design enables the knowledge base to be updated with each new patch without any modification to the underlying model.

Match Data

To adapt the model to high-level play, we collected match data from 100 Diamond-tier players, each with up to 20 recent games, sourced via the Riot Games API. This data provided build and performance data reflective of high-level competitive play. This data captures champion selections, itemization, rune choices, and win rates representative of the top percentile of the player base. This data was used to fine-tune the model on realistic, high-quality in-game decision patterns.

Champion Build and Matchup Statistics (u.gg)

Where the data comes from. Champion build and meta statistics are obtained directly from u.gg’s public stats CDN (<https://stats2.u.gg>). u.gg serves its website from a static JSON tree at this endpoint, so the same files the web client renders can be fetched without a headless browser or scraping HTML. For every (patch, rank, role, champion) tuple we download two documents: an *overview* file containing win rate, pick rate, optimal core and starting items, full rune pages, summoner spells, skill order, and stat shards; and a *matchups* file containing per-opponent win rates and sample sizes (strong-against, weak-against, and full matchup tables).

Numeric IDs for champions, items, runes, and summoner spells are resolved against Riot’s free Data Dragon CDN, so no Riot API key is required for this pipeline. We collected the four most recent patches available on the CDN — at Diamond rank, world region, ranked solo/duo queue.

Why we collected it this way. The official Riot API only exposes raw per-match data. Reproducing u.gg-style aggregates (win rates, optimal builds, matchup tables) from the Riot API would require pulling tens of thousands of matches per (patch, rank) cell and re-implementing the aggregation ourselves, which is well beyond the scope and rate-limit budget of this project. Scraping the pre-computed JSON that u.gg already serves to its own frontend lets us obtain the exact statistics players see in practice, at negligible cost, and at a granularity (per-patch, per-rank, per-role) that aligns directly with how the meta actually shifts. Patch notes alone cannot answer questions such as “*what does Seraphine actually build in Diamond on 16.9?*” or “*which matchups is Kassadin currently losing?*” — these require empirical data, and u.gg’s aggregates are the most practical source of it.

Methods

Retrieval-Augmented Generation

To allow the model to answer questions grounded in current League of Legends data, we built a RAG pipeline combining three data sources:

- **Patch notes** scraped from the official League of Legends website.
- **Champion and item descriptions** fetched from the Riot Data Dragon API.
- **Diamond-tier build data** collected from U.GG, containing recommended runes, summoner spells, core items, skill orders, and win rates per champion and role.

All sources are split into chunks, each annotated with metadata (patch version, section, subject, change type). The chunks are embedded with `sentence-transformers/all-MiniLM-L6-v2` and stored in a FAISS index for fast similarity search.

At query time, we retrieve the top $k = 7$ most relevant chunks using Maximum Marginal Relevance (MMR, $\text{fetch_k} = 20$, $\lambda = 0.6$) to balance relevance with diversity across the different sources. The retrieved chunks, together with their metadata, are concatenated into the system prompt that is fed to Llama-3-8B-Instruct.

Fine-Tuning Data Pipeline

Motivation

While the RAG system retrieves context at query time, it lacks grounding in actual gameplay data — how Diamond-level players build champions, perform, and win or lose in practice.

To address this, we constructed a supervised fine-tuning (SFT) dataset from real ranked match data, following the QLoRA approach demonstrated in the course material.

Data Collection

Match data was collected from Diamond EUW ranked players using the Riot Games API. Player names and tags were scraped from op.gg, converted to PUUIDs via the Riot Account API, and used to fetch match histories. A total of 102 Diamond EUW players were collected, with 20 matches per player, yielding approximately 2,000 matches in total.

For each match, structured fields were extracted including champion, position, patch version, items, keystone rune, summoner spells, KDA, CS, gold, damage dealt, vision score, kill participation, turret kills, game duration, allied team, and enemy team. The resulting records were stored as `match_data.json`.

Synthetic Q&A Generation

Raw match statistics are not directly suitable for language model fine-tuning, which requires natural language instruction-response pairs. We therefore used `meta-llama/Meta-Llama-3-8B-Instruct` — the same model used in the RAG pipeline — to generate natural language answers from each match record.

For every match, four questions were generated covering: item build and runes, performance summary, win/loss analysis, and team composition. The model was prompted with a structured context block derived from the match fields and instructed to respond in 2–4 natural sentences using exact statistics from the data.

Each training example was stored in the format expected by SFTTrainer:

```
### Human: How did Zed perform in this
Jungle game on patch 16.5?

### Context: Champion: Zed | Position:
Jungle | Patch: 16.5 ...

### Assistant: Zed had a dominant
performance, finishing 7/3/14 with
232 CS and 16,070 gold in under 30 minutes.
His 66% kill
participation shows consistent map presence
throughout the game.
```

Fine-Tuning

The base model was fine-tuned on approximately 8000 instruction-response pairs using QLoRA, which freezes the original model weights and trains only a small set of low-rank adapter matrices. This keeps the model’s general language capabilities intact while steering it toward Diamond-level gameplay knowledge — item choices, rune rationales, and performance patterns specific to high-ELO ranked play. Training ran for a single epoch, which was sufficient given that the dataset was

synthetically generated with consistent formatting and low noise.

Qualitative Evaluation

To assess the effect of retrieval and fine-tuning on answer quality, we evaluated five system configurations on a fixed set of 30 questions covering four categories: patch notes, leaderboard rankings, champion descriptions, and Diamond-tier build data.

Systems Evaluated

- **A: Base** – Llama-3-8B-Instruct with no retrieval and no fine-tuning.
- **B: Fine-tuned** – Llama-3-8B-Instruct with our LoRA adapter trained on Diamond-tier match data, no retrieval.
- **C: Base + RAG** – Base Llama-3-8B-Instruct with retrieval enabled.
- **D: Fine-tuned + RAG** – Our full system: fine-tuned model with retrieval enabled.
- **E: Llama-3.3-70B (reference)** – A significantly larger model accessed via the Groq API, used without retrieval as an external strong baseline.

Each system produced an answer for every question, and ground-truth references were prepared in advance based on the original data sources (patch notes, U.GG build statistics, Riot Data Dragon). We used a two-stage evaluation:

1. Automated LLM-as-Judge. Each (question, ground truth, candidate answer) triple was scored by Llama-3.3-70B (served via the Groq API) on two axes: *correctness* (factual agreement with the ground truth) and *relevance* (whether the answer actually addresses the question), each on a 0–5 scale. The judge returned a structured JSON verdict per answer, which we aggregated per system and per category.

2. Human Evaluation. To validate the automated scores, two League of Legends players independently scored all $5 \times 30 = 150$ answers using the same correctness and relevance rubric. Their judgements were averaged and used as the primary evaluation reported in the results.

Results

Tables 1 and 2 report the overall correctness and relevance of all five systems. Table 1 shows the raw scores from the automated LLM judge (Llama-3.3-70B via Groq), while Table 2 reports the human scores together with 95% bootstrap confidence intervals (1000 iterations) over the per-question averages of two annotators.

Retrieval is the decisive factor. The single largest gain in answer quality comes from giving the model access to the indexed knowledge base. Under human evaluation, correctness more than **doubles** when retrieval is added to a base model — questions like “*what is the recommended core build*

Table 1. Automated LLM-judge scores across 30 questions. A: Base, B: Fine-tuned, C: Base+RAG, D: Fine-tuned+RAG, E: Llama-3.3-70B (reference).

System	Correctness	Relevance	Combined
A	1.30	4.33	2.82
B	1.60	4.10	2.85
C	2.60	4.60	3.60
D	2.87	4.63	3.75
E	2.60	4.83	3.72

Table 2. Human evaluation with 95% bootstrap CIs (1000 iterations). Format: **mean [2.5%, 97.5%]**. A: Base, B: Fine-tuned, C: Base+RAG, D: Fine-tuned+RAG, E: Llama-3.3-70B (reference).

System	Correctness	Relevance	Combined
A	2.13 [1.82, 2.47]	3.43 [3.05, 3.82]	2.78 [2.48, 3.07]
B	2.35 [2.05, 2.67]	3.75 [3.42, 4.05]	3.05 [2.80, 3.31]
C	3.88 [3.50, 4.23]	4.53 [4.28, 4.75]	4.21 [3.90, 4.47]
D	4.02 [3.68, 4.35]	4.48 [4.25, 4.68]	4.25 [3.97, 4.49]
E	3.08 [2.77, 3.40]	3.77 [3.50, 4.02]	3.42 [3.20, 3.67]

for *Veigar Mid?*” or “*which junglers have the highest win rate this patch?*” require facts that simply do not exist in the model’s pretraining data. Without RAG, the base model either refuses, generalizes from outdated patches, or fabricates plausible-sounding but incorrect builds; with RAG, it cites the actual Diamond-tier statistics retrieved from u.gg. The confidence intervals for the two RAG systems do not overlap with any of the non-RAG systems, so this effect is statistically meaningful given our sample of 30 questions and not an artifact of which questions we happened to pick.

Fine-tuning helps the model use the retrieved context. On its own, fine-tuning on Diamond match data provides only a small correctness gain (2.13 → 2.35), with overlapping confidence intervals; the LoRA adapter learns the *style* of Diamond-level analysis (talking about KDA, kill participation, core item timing) but cannot make up for missing facts. Its real value emerges when paired with retrieval: the fine-tuned + RAG system achieves the highest combined score (4.25) and noticeably better relevance than base + RAG on build-data questions, because the adapter has learned to phrase answers in the language of competitive play rather than as generic Q&A. The improvement over base + RAG is small in magnitude but consistent across categories, indicating that fine-tuning makes the retrieved context easier for the model to integrate, even when the raw factual content is the same.

Retrieval beats raw model size. The 70B reference model achieves a respectable correctness of 3.08 from its broad pre-training knowledge alone — it can correctly describe well-known champions, popular item builds, and common matchups it has seen during pretraining. However, it has no access to latest patch win rates, no awareness of the current meta, and no insight into Diamond-tier builds, so it loses badly on patch-specific questions (e.g., current Jungle win rates, recent nerfs, post-rework item priorities). Our 8B fine-tuned + RAG sys-

tem outperforms the 70B reference by **31%** in correctness with non-overlapping confidence intervals, despite using a backbone roughly nine times smaller. In a domain where the ground truth changes every two weeks, a current and well-indexed knowledge base is more valuable than additional model parameters.

Cross-checking the LLM judge against humans. Both evaluation methods produce the same overall ranking — RAG systems on top, the 70B reference in the middle, non-RAG 8B baselines at the bottom — which gives us some confidence that the automated judge is tracking real quality differences and not just surface features. The main disagreement is on *relevance*: the automated judge tends to be generous to fluent, well-structured answers even when they are off-topic, while human evaluators penalize answers that sound confident but talk about the wrong champion, the wrong role, or an outdated patch. This bias is most visible on the non-RAG systems and the 70B reference, all of which produce smooth, plausible-sounding text. The human numbers should therefore be treated as the primary result; the LLM judge is useful as a cheap, fast sanity check, but it consistently over-rewards stylistic polish over factual grounding.

Conclusion

This project showed that a small open-weight language model, equipped with the right retrieval pipeline and lightly adapted to its domain, can outperform a much larger general-purpose model on a fast-moving topic like League of Legends. Across both automated and human evaluations, retrieval emerged as the dominant driver of factual accuracy, while fine-tuning on Diamond-tier match data added a smaller but consistent benefit by teaching the model how to phrase and integrate the retrieved context. The full fine-tuned + RAG system clearly outperformed a 9× larger reference model with non-overlapping confidence intervals, demonstrating that for specialized, rapidly-changing domains a current and well-organized knowledge base is more valuable than additional model parameters. Just as importantly, when the game updates, only the indexed data needs to be refreshed — the model itself does not.

Future Work

A next step could be to scale the fine-tuning corpus well beyond the current single-patch, Diamond-only snapshot — ideally aggregating match data across multiple seasons and rank tiers, so the model learns how builds, runes, and matchups evolve as the meta shifts. With a substantially larger and more diverse dataset, the fine-tuned model could begin to reason about *why* certain builds emerge in certain patches, not just *what* they are. Pairing this stronger backbone with the existing retrieval layer — and ideally training the two jointly — would push the system from answering questions to actually explaining the game.

References

- [1] Ruben Ferreira and Jana Faganeli Pucer. Quantifying player death impact in league of legends. In *Proceedings of the 11th Student Computing Research Symposium*, pages 51–54, 2025.
- [2] Philip Z Maymin. Smart kills and worthless deaths: esports analytics for league of legends. *Journal of Quantitative Analysis in Sports*, 17(1):11–27, 2021.
- [3] Lincoln Magalhaes Costa, Rafael Gomes Mantovani, Francisco Carlos Monteiro Souza, and Geraldo Xexéo. Feature analysis to league of legends victory prediction on the picks and bans phase. In *2021 IEEE conference on games (CoG)*, pages 01–05. IEEE, 2021.
- [4] Julie Jiang, Kristina Lerman, and Emilio Ferrara. Individualized context-aware tensor factorization for online games predictions. In *2020 International Conference on Data Mining Workshops (ICDMW)*, pages 292–299. IEEE, 2020.
- [5] Jungmin Lee, Inhee Cho, and Youngjae Yoo. Leaguebot: A voice llm companion of cognitive and emotional support for novice players in competitive games. *arXiv preprint arXiv:2602.01213*, 2026.
- [6] Riot Games. Riot games developer portal. Accessed: 2026-03-20.